

Name: Krishna Raviraj Shimpi
PRN No: 202501040036

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators

02:59

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations :

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations :

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations :

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

```
import numpy as np
```

```
def array_operations(A, B):
```

```
# Convert A and B to NumPy arrays
```

```
    A = np.array(A)
```

```
    B = np.array(B)
```

```
    # Arithmetic Operations
```

```
    sum_result = A + B
```

```
    diff_result = A - B
```

```
    prod_result = A * B
```

```
    # Statistical Operations
```

```
    mean_A = np.mean(A)
```

```
    median_A = np.median(A)
```

```
    std_dev_A = np.std(A)
```

```
    # Bitwise Operations
```

```
    and_result = np.bitwise_and(A, B)
```

```
    or_result = np.bitwise_or(A, B)
```

```
    xor_result = np.bitwise_xor(A, B)
```

```
# Output results with one space between each element
```

```
    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
```

```
    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
```

```
print("Element-wise Product:", ' '.join(map(str, prod_result)))
```

```
print(f"Mean of A: {mean_A}")
```

```
print(f"Median of A: {median_A}")
```

```
print(f"Standard Deviation of A: {std_dev_A}")
```

```
print("Bitwise AND:", ' '.join(map(str, and_result)))
```

```
print("Bitwise OR:", ' '.join(map(str, or_result)))
```

```
print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

```
A = list(map(int, input().split())) # Elements of array A
```

```
B = list(map(int, input().split())) # Elements of array B
```

```
array_operations(A, B)
```

The screenshot displays a code execution interface with a sidebar on the left labeled 'Explorer' and a sidebar on the right labeled 'Debugger'. The main area shows the execution results for a function named `array_operations`. At the top, there are two summary boxes: 'Average time' showing '0.013 s' (13.00 ms) and 'Maximum time' showing '0.018 s' (18.00 ms). To the right of these, a green checkmark indicates '1 out of 1 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. Below this, a section for 'Test case 1' (18 ms) is shown with a 'Debug' button. The test case details are presented in a table with two columns: 'Expected output' and 'Actual output'. The expected output shows two input arrays: `1 2 3 4` and `5 6 7 8`. The actual output shows the same two arrays. Below the arrays, the expected output lists several calculations: 'Element-wise Sum: 6 8 10 12', 'Element-wise Difference: -4 -4 -4 -4', 'Element-wise Product: 5 12 21 32', 'Mean of A: 2.5', 'Median of A: 2.5', 'Standard Deviation of A: 1.118033988749895', 'Bitwise AND: 1 2 3 0', 'Bitwise OR: 5 6 7 12', and 'Bitwise XOR: 4 4 4 12'. The actual output shows the same results for all these calculations.

Expected output	Actual output
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
Element-wise Sum: 6 8 10 12	Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4	Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32	Element-wise Product: 5 12 21 32
Mean of A: 2.5	Mean of A: 2.5
Median of A: 2.5	Median of A: 2.5
Standard Deviation of A: 1.118033988749895	Standard Deviation of A: 1.118033988749895
Bitwise AND: 1 2 3 0	Bitwise AND: 1 2 3 0
Bitwise OR: 5 6 7 12	Bitwise OR: 5 6 7 12
Bitwise XOR: 4 4 4 12	Bitwise XOR: 4 4 4 12